

Building a POST/PATCH/DELETE API Lab (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-building-post-patch-delete-api-lab>  <https://github.com/learn-co-curriculum/python-p4-building-post-patch-delete-api-lab/issues/new>

Learning Goals

- Build an API to handle GET, POST, PATCH, and DELETE requests.

Key Vocab

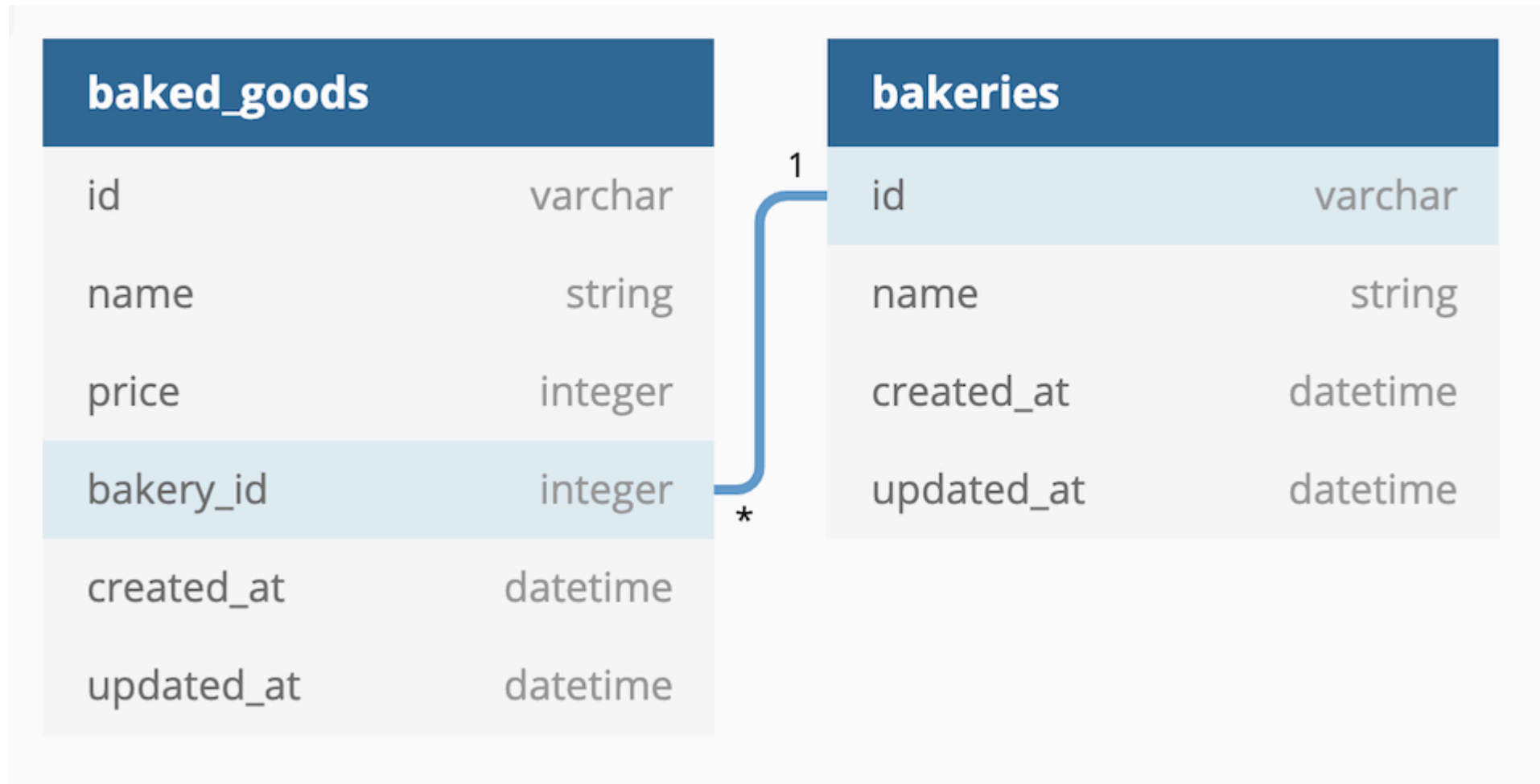
- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.

Instructions

This is a **test-driven lab**. Run `pipenv install` to create your virtual environment and `pipenv shell` to enter the virtual environment. Then run `pytest -x` to run your tests. Use these instructions and `pytest` 's error messages to complete your work in the `server/` folder. Make sure to test your routes in Postman as you progress.

```
$ pipenv install  
$ pipenv shell
```

In this application, we'll be working on a familiar JSON API to get a list of bakeries and their baked goods. We have two models, bakeries and baked goods, in a one-to-many relationship. The migrations are already set up. Here's a reminder of what the ERD for these tables looks like:



You can pick up where we left off by entering the following commands:

```
$ cd server
$ flask db upgrade
$ python seed.py
$ python app.py
```

Edit `server/app.py` to support the following requests:

- Define a `POST` block inside of a `/baked_goods` route that creates a new baked good in the database and returns its data as JSON. The request will send data in a form.
- Define a `PATCH` block inside of the `/bakeries/<int:id>` route that updates the name of the bakery in the database and returns its data as JSON. As with the previous `POST` block, the request will send data in a form. The form does not need to include values for all of the bakery's attributes.
- Define a `DELETE` block inside of a `/baked_goods/<int:id>` route that deletes the baked good from the database and returns a JSON message confirming that the record was successfully deleted.

Once all of your tests are passing, commit and push your work using `git` to submit.

Examples

`POST /baked_goods`

post patch delete bakery lab / **post baked_goods** 

POST  localhost:5555/baked_goods **Send** 

Params Auth Headers (8) **Body**  Pre-req. Tests Settings Cookies

form-data 

	Key	Value	Description	...	Bulk Edit
☒	name	Giant Donut Hole			
☒	price	12			
☒	bakery_id	1			
	Key	Value	Description		

Body  201 CREATED 15 ms 449 B [Save Response](#) 

Pretty Raw Preview Visualize JSON  

```
1  {
2    "bakery": {
3      "created_at": "2023-11-03 18:42:41",
4      "id": 1,
5      "name": "Your Bakery",
6      "updated_at": "2023-11-03 18:43:14"
7    },
8    "bakery_id": 1,
9    "created_at": "2023-11-03 18:48:04",
10   "id": 5,
11   "name": "Giant Donut Hole",
12   "price": 12,
13   "updated_at": null
14 }
```

PATCH /bakeries/<int:id>

PATCH

localhost:5555/bakeries/1

Send

Params

Auth

Headers (8)

Body

Pre-req.

Tests

Settings

form-data

DELETE /baked_goods/<int:id>

post patch delete bakery lab / delete baked goods

DELETE localhost:5555/baked_goods/1 Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies

none

This request does not have a body

Body 200 OK 11 ms 212 B Save Response

Pretty Raw Preview Visualize JSON

```
1  
2  "message": "record successfully deleted"  
3
```

NOTE: You can use Postman to make get requests for bakeries and baked_goods before and after your post, patch, and delete requests to ensure the correct record is being added, updated, or deleted.

Resources

- [Flask - Pallets](https://flask.palletsprojects.com/en/2.2.x/)
- [POST - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST)
- [PATCH - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/PATCH)
- [DELETE - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/DELETE)
- [SQLAlchemy-serializer - PyPI](https://pypi.org/project/SQLAlchemy-serializer/)

This tool needs to be loaded in a new browser window

Load Building a POST/PATCH/DELETE API Lab (CodeGrade) in a new window